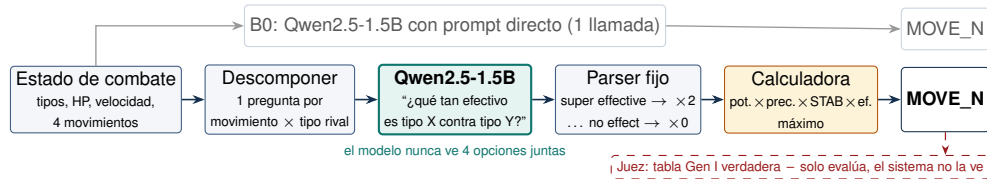


Descomponer para decidir: un modelo de 1,5B juega Pokémon Blue

Deliverable 2 — del diagnóstico a un sistema que funciona

Generative Artificial Intelligence (580694) • Primavera 2026 • UdeC

Equipo: **Pedro Dañobeytia • Rafael Fernández • Cristian González • Daniel González** Repo: github.com/Rafaefe/AIBattlesInPokemon Video: https://www.youtube.com/watch?v=IrEU_m0MMA



1. CONTINUIDAD CON EL D1

Misma tarea, mismo fallo, mismo criterio. Estado de combate → MOVE_1–MOVE_4; correcto = máximo de potencia × precisión × STAB × efectividad (Gen I). Fallo del D1: con prompt estructurado acertaba 16,7%, *bajo el azar*: respondía por posición en vez de combinar los atributos.

Desviaciones declaradas. (i) B0 = prompt directo del D1 + pedir MOVE_N (amenaza T1 del D1). (ii) **50** estados, mismo generador y semilla (20260830; con $n=30$ reproduce los del D1) + **50 nuevos** (semilla 20260927).

2. MODELO ELEGIDO

Qwen/Qwen2.5-1.5B-Instruct, revisión 989aa7980e, 1,54 B parámetros, Colab Tesla T4, float32 (torch 2.11.0+cu128, transformers 5.16.1). B0, A1, A2, A3, S1, S1⁺ (50 originales) corrieron antes en CPU, float32.

- **Economía:** el más pequeño de los tres candidatos del D1 y el único con un perfil de fallo *medido* en esta tarea.
- **Le quita el paso que lo separaba de los otros dos**, razonar en varios pasos (GSM8K: Phi-3.5-mini 86,2, Llama-3.2-3B 77,7, Qwen 73,2): aquí combina la calculadora.

3. LA INTERVENCIÓN Y SU VÍNCULO CON EL FALLO

Descomposición + herramienta (diagrama): una pregunta por movimiento usable y tipo rival; el modelo responde con las frases del juego, un parser fijo las traduce a $\times 2 / \times 1 / \times 0,5 / \times 0$ y la calculadora elige el máximo.

- **Ataca el sesgo de posición:** el modelo nunca ve las cuatro opciones juntas.
- **Ataca la combinación:** el modelo aporta hechos sueltos; la aritmética la hace código.
- **El modelo es la única fuente de tipos:** la solución no tiene la tabla (lo verifica un test). Referencia **H**: la misma calculadora con toda efectividad en $\times 1$.

4. ESTRATEGIAS EVALUADAS ($n = 50$, MISMOS ESTADOS)

	Qué cambia	Ilam.	ACC [IC 95%]	vs B0
B0	prompt directo (baseline)	1,0	28,0 % [18–42]	–
A1	prompt estructurado del D1	1,0	20,0 % [11–33]	+3/–7
A2	B0 con 4 órdenes + voto	4,0	30,0 % [19–44]	+2/–1
A3	chain-of-thought + fórmula	1,0	28,0 % [18–42]	+3/–10
S1	hechos en números (v1)	5,2	58,0 % [44–71]	+17/–2
S1 ⁺	S1 calibrada	5,2	42,0 % [29–56]	+11/–4
H	sin modelo: ignora tipos	0	54,0 % [40–67]	+16/–3
S2	hechos en palabras	5,2	48,0 % [35–62]	+13/–3

“vs B0”: estados que gana/pierde contra el baseline en el mismo estado; S2 vs B0, McNemar exacto $p = 0,021$. A3 escribía a veces el nombre del movimiento en vez de MOVE_N: se interpreta igual que en el D1 (STRICT 56%).

5. Fallo explicado de la v1: S1 no usaba el modelo

S1 preguntaba el multiplicador en números (2/1/0,5/0). El modelo respondió $\times 0,5$ en 122/122 pares (prob. media 0,97), incluso Agua contra Fuego, y $\times 0,5$ con prob. 0,90 ante una pregunta vacía: decidía el formato, no los tipos. S1 se reduce a H (**elige lo mismo en 48 de 50 estados**): su 58,0 % no es mérito del modelo. De ahí S2.

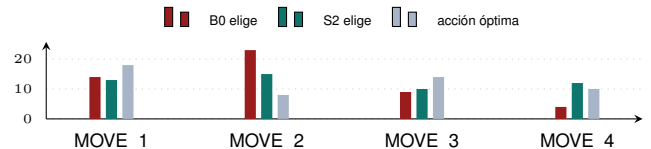
Resultado en breve

S2 corrige el fallo del D1 (sin sesgo de posición) y **supera al baseline**: 28,0 % → **48,0 %** en los 50 originales ($p = 0,021$) y 40,0 % → **58,0 %** en 50 nuevos ($p = 0,064$). Pero destapa un fallo más profundo: **el modelo no sabe la tabla de tipos**, y H, que la ignora, acierta 54,0 % y 76,0 %.

6. CONFIRMACIÓN EN 50 ESTADOS NUEVOS

	50 originales	50 nuevos
B0 baseline	28,0 % [18–42]	40,0 % [28–54]
H sin modelo	54,0 % [40–67]	76,0 % [63–86]
S2 solución	48,0 % [35–62]	58,0 % [44–71]

En los estados nuevos (misma sesión y GPU) S2 gana a B0 en 14 y pierde en 5 ($p = 0,064$). Por dificultad en los originales (EASY-MEDIUM-HARD, 17/17/16): B0 6/17 · 6/17 · 2/16; S2 8/17 · 7/17 · 9/16.



Sesgo de posición: con las 4 rotaciones de A2, B0 eligió la *misma casilla* en las 4 en 0 estados y el *mismo movimiento* en 8 (de 49). χ^2 vs uniforme: B0 15,8 ($p=0,001$), S2 1,0 ($p=0,792$).

7. Caso real donde S2 falla: estado 32 (MEDIUM), Water vs Ground

mov.	tipo	efectividad		score	
		modelo	Gen I	S2	real
MOVE_1	Thunderbolt	Electric	$\times 0$	95,0	0,0
MOVE_2	Strength	Normal	$\times 1$	80,0	80,0
MOVE_3	Razor Leaf	Grass	$\times 2$	52,2	104,5
MOVE_4	Psybeam	Psychic	$\times 1$	65,0	65,0

Eligió MOVE_1 (Thunderbolt); el óptimo era MOVE_3 (Razor Leaf) (pierde el 100 % del mejor score). **Por qué:** ante Electric → Ground el modelo dijo “normal damage” ($\times 1$); en Gen I es $\times 0$. Es un hecho de la tabla que el modelo no sabe. Con los hechos correctos la misma calculadora acierta: sí.

8. LÍMITES CONOCIDOS

- **El modelo no aporta sobre ignorar los tipos:** frente a H, S2 gana en 2 estados y pierde en 5 ($p = 0,453$). En los 50 estados nuevos es *peor*: gana en 0, pierde en 9 ($p = 0,004$). Sus hechos de tipos valen menos que asumir $\times 1$.
- **Por qué: el modelo no sabe la tabla de tipos.** Respondió “normal damage” en 106/122 pares (87 %); cuando dijo otra cosa acertó 2/16 (nuevos: 1/12) y llamó *super effective* a pares del mismo tipo (Eléctrico–Eléctrico, Veneno–Veneno, Psíquico–Psíquico, Roca–Roca, Agua–Agua) que en Gen I no lo son. Responder siempre $\times 1$ acierta 80/122 pares; el modelo, 74/122. Por efectividad real: $\times 2$ 2/18, $\times 1$ 72/80, $\times 0,5$ 0/21, $\times 0$ 0/3. Ninguno de sus 48 errores es el valor de Gen II+: no confunde generaciones.
- **Costo:** 5,2 preguntas por estado vs 1 de B0; en la T4, 0,10 vs 0,33 s por estado (respuestas cortas; cada par se pregunta una vez). Como en el D1, solo daño inmediato: no usa HP, velocidad ni estados alterados.
- **Qué sigue (D3):** el cuello de botella pasó de *combinar a saber*: enseñarle la tabla (ajuste fino); darla por recuperación haría innecesario al modelo en este paso.

Reproducir: `python -m d2.evaluate [--holdout] • python -m d2.demo [--replay 32] • cifras desde results_d2*/ con python report/fill_d2.py`